

Secure Exit

SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

CO3-AT3-Assignment

[← Back](#)

QUESTIONS

Q1

Banking Transaction System –
Exception Handling
10 marks

Q2

Hospital Patient Registration –
User-Defined Exception
10 marks

Q3

E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4

Hospital Emergency System –
Thread Priority
10 marks

Q5

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

1/5 answered

Q1

Banking Transaction System –
Exception Handling10
marks

Develop a Java program for an ATM withdrawal system. The program accepts the account balance and withdrawal amount.

The program must:

- Accept the current account balance and withdrawal amount.
- Throw a user-defined `InsufficientBalanceException` when the withdrawal amount exceeds the balance.
- Handle withdrawal amounts that are zero or negative.
- Handle invalid numeric input using a suitable built-in exception.
- Display a meaningful transaction result.
- Use try-catch-finally so that the transaction completion message is always displayed.

Sample Test Cases:

Input / Scenario	Expected Result	Purpose
10000 3000	Normal case	
10000 12000	Boundary / exception case	
10000 -500	Invalid input	
abc 3000	Invalid input	

Your Code

```
1 import java.util.*;
```

```
2 public class Main{
3     public static void main(String[] args)
4         Scanner sc=new Scanner(System.in);
5         try{
6             double balance=sc.nextDouble()
7             double amount=sc.nextDouble();
8             if (amount<=0){
9                 throw new InsufficientBalanc
10            }
11            if(amount>balance){
12                throw new InsufficientBalanc
13            }
14            balance-=amount;
15            System.out.println("Withdrawal
16            System.out.println("Remaining
17        }
18        catch(InputMismatchException e){
19            System.out.println("Invalid in
20        }
21        catch(InsufficientBalanceException
22            System.out.println(e.getMessag
23        }
24
25        finally{
26            System.out.println("Transction
27        }
28        sc.close();
29    }
30 }
31 class InsufficientBalanceException extends
32     public InsufficientBalanceException (St
33         super (message) ;
34     }
35 }
```



Input

10000
3000

Output

```
Withdrawal successful  
Remaining balance: 7000.0  
Transction completed
```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.

[Secure Exit](#)

CO3-AT3-Assignment

[← Back](#)

QUESTIONS

Q1

Banking Transaction System –
Exception Handling
10 marks

**Q2**

Hospital Patient Registration –
User-Defined Exception
10 marks

**Q3**

E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4

Hospital Emergency System –
Thread Priority
10 marks

Q2

Hospital Patient Registration – User-Defined Exception

10
marks

Develop a Java program for a hospital patient registration system. The program accepts the patient's name, age, and body temperature. Create suitable user-defined exception classes for invalid age and invalid temperature. Validate the input before registration. Use try-catch-finally and display a registration completion message from finally.

Sample Test Cases:

Input / Scenario	Expected Result	Purpose
Arun, 35, 98.6	Patient registered successfully	Normal case
Meena, 12, 98.4	InvalidAgeException	Boundary / exception case



SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING



MD Imthiaz I
192521360RD

```
4 Scanner sc=new Scanner(System.in);  
5 try{  
6     String name=sc.next();
```

```
7         int age=sc.nextInt();
8         double temp=sc.nextDouble();
9         if(age<18){
10             throw new InvalidAgeExcepti
11         }
12         if (temp<95||temp>100){
13             throw new InvalidTemperatu
14         }
15         System.out.println("Patient re
16     }
17     catch(InvalidAgeException e){
18         System.out.println(e.getMessag
19     }
20     catch(InvalidTemperatureException
21         System.out.println(e.getMessag
22     }
23     finally{
24         System.out.println("Registrati
25     }
26     sc.close();
27 }
28 }
29 class InvalidAgeException extends Exceptio
30     InvalidAgeException(String msg){
31         super(msg);
32     }
33 }
34 class InvalidTemperatureException extends
35     InvalidTemperatureException(String msg)
36         super(msg);
37     }
38 }
```



Input

Arun
35
98.6

Output

Patient registered successfully
Registration completed

✔ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.

[Secure Exit](#)**CO3-AT3-Assianment**[← Back](#)

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RDUser-Defined Exception
10 marks**Q3**E-Commerce Order Processing
– Built-in Exceptions
10 marks**Q4**Hospital Emergency System –
Thread Priority
10 marks**Q5**Online Food Delivery – Multiple
Threads and Execution Order
10 marks

3/5 answered

The program must:

- Calculate total cost as quantity × price.
- Handle invalid product indices.
- Handle invalid numeric input.
- Reject zero or negative quantities.
- Use appropriate built-in exception classes.
- Continue execution after handling an invalid transaction.

Sample Test Cases:

Input / Scenario Expected Result Purpose

1, 2 Valid product; total cost calculated Normal case

5, 2 ArrayIndexOutOfBoundsException handled

Boundary / exception case

2, -1 Invalid quantity Invalid input

abc, 2 InputMismatchException handled Invalid input

Your Code

```
1 import java.util.Scanner;  
2 import java.util.InputMismatchException;  
3 public class Main{  
4     public static void main(String[] args)
```

```
5      Scanner sc=new Scanner(System.in);
6      String[] products={"Laptop","Phone"};
7      double[] prices={50000,30000,2000,1000};
8      try{
9          int index=sc.nextInt();
10         int quantity=sc.nextInt();
11         if(quantity<=0){
12             System.out.println("Invalid quantity");
13         }else{
14             double total=prices[index]*quantity;
15             System.out.println("Product: "+products[index]);
16             System.out.println("Total cost: "+total);
17         }
18     }
19     catch(InputMismatchException e){
20         System.out.println("Invalid input");
21     }
22     sc.close();
23 }
24 }
```

Input

1
2

Output

Product: Phone
Total cost: 60000.0

✓ Submitted

[Secure Exit](#)

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

Exception Handling

10 marks

Q2Hospital Patient Registration –
User-Defined Exception

10 marks

**Q3**E-Commerce Order Processing
– Built-in Exceptions

10 marks

**Q4**Hospital Emergency System –
Thread Priority

10 marks

**Q5**Online Food Delivery – Multiple
Threads and Execution Order

10 marks

4/5 answered

A hospital application creates threads for Emergency Patient Processing, General Patient Processing, and Billing Processing.

Develop a Java program that assigns different priorities to these threads. Run the program several times and analyze whether the emergency thread is guaranteed to execute first.

Then modify the design if the hospital requires a strict execution sequence rather than merely expressing scheduling preference.

Your answer must contain Java code and an execution analysis based on the actual thread behavior.

Sample Test Cases:

Input / Scenario	Expected Result	Analysis	Focus
Emergency, General, Billing	All threads execute;	priority does not guarantee a fixed output order	Normal execution

Emergency workload > General workload
Emergency thread receives a higher priority but exact ordering is not guaranteed
Concurrency / design
Strict emergency-before-billing requirement
Use

explicit coordination rather than relying only on
priority Exception / boundary

Your Code

```
1 public class Main{
2     public static void main(String[] args)
3         Emergency e=new Emergency();
4         General g=new General();
5         Billing b=new Billing();
6         e.setPriority(Thread.MAX_PRIORITY);
7         e.setPriority(Thread.NORM_PRIORITY);
8         e.setPriority(Thread.MIN_PRIORITY);
9         e.start();
10        g.start();
11        b.start();
12        e.join();
13        g.join();
14        b.join();
15        System.out.println("All processing completed");
16    }
17 }
18 class Emergency extends Thread{
19     public void run(){
20         System.out.println("Emergency Patient");
21     }
22 }
23 class General extends Thread{
24     public void run(){
25         System.out.println("General Patient");
26     }
27 }
28 class Billing extends Thread{
29     public void run(){
30         System.out.println("Billing Process");
31     }
32 }
```

Input

stdin input

Output

```
Emergency Patient Processing  
Billing Processing  
General Patient Processing  
All processing completed
```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.

Secure Exit

SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

CO3-AT3-Assignment

[← Back](#)

QUESTIONS

Q1

Banking Transaction System –
Exception Handling
10 marks

Q2

Hospital Patient Registration –
User-Defined Exception
10 marks

Q3

E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4

Hospital Emergency System –
Thread Priority
10 marks

Q5

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

Q5

**Online Food Delivery – Multiple Threads
and Execution Order**10
marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus
Order=Pizza; Payment=UPI; Delivery=Agent A Order
→ Payment → Delivery Normal execution
Order=Burger; Payment=Card; Delivery=Agent B

Order → Payment → Delivery Concurrency / design
Payment=Failed Delivery assignment should not
proceed Exception / boundary

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args)
4     {
5         Scanner sc=new Scanner(System.in);
6         String input=sc.next();
7         String[] data=input.split(";");
8         String order=data[0].split("=")[1];
9         String payment=data[1].split("=")
10        String delivery = data[2].split("=")
11        OrderThread o=new OrderThread();
12        PaymentThread p=new PaymentThread
13        o.start();
14        o.join();
15        p.start();
16        p.join();
17        DeliveryThread d=new DeliveryThread
18        d.start();
19        d.join();
20        sc.close();
21    }
22 }
23 class OrderThread extends Thread {
24     public void run(){
25         System.out.println("Order process:");
26     }
27 }
28 class PaymentThread extends Thread{
29     boolean success;
30     PaymentThread(String payment){
31         success=!payment.equalsIgnoreCase
32     }
33     public void run(){
34         if (success){
35             System.out.println("Paymer");
36         }
37         else{
38             System.out.println("Paymer");
39         }
40     }
41 }
42 class DeliveryThread extends Thread{
```

```
42     boolean paymentSuccess;  
43     DeliveryThread(boolean status) {  
44         paymentSuccess=status;  
45     }  
46     public void run() {  
47         if (paymentSuccess) {  
48             System.out.println("Delivery a  
49         }  
50         else{  
51             System.out.println("Delivery a  
52         }  
53     }  
54 }
```



Input

Order=Pizza;Payment=UPI;Delivery=Agent A

Output

```
Order processinnng completed  
Payment processing completed  
Delivery assigned
```

✓ Submitted